

Closure-Based Syntax for Contracts

Document #: P2461R0
Date: 2021-10-14
Project: Programming Language C++
Audience: WG21 SG21 (Contracts)
Reply-to: Gašper Ažman
<gasper.azman@gmail.com>
Caleb Sunstrum
<calebs@edg.com>
Bronek Kozicki
<brok@spamcop.net>

Contents

1	Introduction	2
1.1	On Extensions and Viability	2
2	Proposal	2
2.1	Example	2
2.2	Proposed syntax	3
2.2.1	MVP Restrictions	3
3	Future Extensions (not a proposal)	4
3.1	Example	4
4	Semantics	4
4.1	Evaluation order	4
4.1.1	Assertions	4
4.1.2	pre- and post-conditions	5
4.2	post-condition reference-capture limitations in the MVP	5
4.3	Side-effect elision	6
5	New good stuff if we pick closure-based semantics	6
5.1	Stateful contracts (extension)	6
5.2	Destructuring the return value	6
5.3	We can have attributes appertaining to contract annotations	7
6	Challenges with the attribute-derived syntax	7
6.1	Place for annotations like “axiom”, “new”, etc.	7
6.2	Referencing function arguments in postconditions	7
6.3	Introducing the return variable	8
6.4	Preconditions and assertions that need copies	9
6.5	Postconditions that need destructuring [when lambda-captures get it]	9
6.6	Multithreaded Usage	9
6.7	Summary	10
7	Mutation and Static Analyzers	10
8	What about abbreviated lambdas?	10

9 C-compatibility	10
10 Proposed Wording	11
11 Acknowledgements	11
12 References	11

1 Introduction

The attribute-derived syntax for contracts is limiting and steps on the shared space between C and C++. This paper explores an alternative syntax that should hopefully be more powerful while being able to express the same semantics.

This paper proposes almost the same semantics as [P2388R2].

The only significant change from [P2388R2] is the semantics of effect elision - this paper specifies it as all-or-nothing, per *correctness-annotation*.

Due to the way this paper models annotations, it may leave fewer things undefined compared to [P2388R2], despite the fact that that it does not propose explicit closures yet.

Note: this paper is an exploration. The authors do not object to the currently agreed-upon syntax; but it does seem to present certain challenges that this paper tries to address.

Note: WG14 has communicated that their vendors don't have a blocking problem with the attribute-like syntax, though they have reservations; in addition, WG21 members have expressed difficulties with teaching the *: means it's not an attribute* intricacies.

1.1 On Extensions and Viability

The authors believe that **any MVP must clearly show plausible syntax for all known extensions**. This does not mean *propose*. It means *show*. Specifying precise semantics for the entire extension space is not in the spirit of a *minimum viable product*, but *viability* implies that all desired features can at some point be supported. This means there must be syntax, so syntax we show.

2 Proposal

2.1 Example

We introduce three context-sensitive keywords: **pre**, **post**, and **assert**. **pre** and **post** are only keywords in the toplevel context of a function declarator. In the future, we can put other keywords between the keyword and the colon (see example).

pre and **post** can appear in function declarations after the place for the optional trailing **requires** clause.

Example:

```
auto plus(auto const x, auto const y) -> decltype(x + y)
    pre { x > 0 }
    pre {
        /* check for overflow - badly */
        (x > 0 && y > 0 ? as_unsigned(x) + as_unsigned(y) > as_unsigned(x) : true) &&
        // since these are conditional-expressions, use '||' to combine them
        (x < 0 && y < 0 ? as_unsigned(x) + as_unsigned(y) < as_unsigned(x) : true)
```

```

}
// ret is as-if auto&&
post (ret) { ret == (x + y) }
{
  assert { x > 0 }; // this is currently "valid" syntax,
                  // but we should reclaim it.
  auto cx = x;
  return cx += y;
}

```

One may note that this is strikingly similar to the syntax proposed in [N1962], way back in 2006. Our thanks to Andrzej Krzemiński for digging this up.

2.2 Proposed syntax

Let's take a look at the generic syntax of a *correctness-annotation* (to use the term from [P2388R2]):

correctness-specifier:

correctness-specifier-keyword *correctness-specifier-introducer*_{opt} *correctness-specifier-body*

correctness-specifier-keyword:

pre | **post** | **assert**

correctness-specifier-introducer:

*lambda-introducer*_{opt} *return-value-id*_{opt}

return-value-decl:

(*identifier*)

correctness-specifier-body:

{ *conditional_expression* }

For the MVP, the *lambda-introducer* is required to be omitted.

If the *lambda-introducer* is omitted, the *correctness-specifier-body* behaves as-if the *lambda-introducer* was [&].

If the *correctness-specifier-keyword* of the *correctness-specifier* is **post**, the *return-value-decl* must be present, and introduces the name for the prvalue or the glvalue result object of the function. This identifier is valid within the *correctness-specifier-body*.

2.2.1 MVP Restrictions

Naming a non-const value parameter in a post-condition is ill-formed for now. This can be lifted by allowing copy-capture later, when we allow the *lambda-introducer* to appear. This is to both prevent **referencing moved-from objects**, and to **allow the calling code to reason** about the properties of the result object, such as in the example:

```

int min(int x, int y)
  post (r) { r <= x && r <= y };

```

The lambda-closure definition works with this - the function arguments are captured by reference, which happens to be reference-to-const, given that they are const, which gives the exact semantics of [P2388R2].

3 Future Extensions (not a proposal)

3.1 Example

The above really just assumes the default capture is `[&]`, which we can introduce later.

Modeling using lambda-captures allows us to explain why post-conditions can't refer to rvalue-reference arguments, and all the other possible implementation-limitations as well.

Example: (extensions not proposed here)

```
auto plus(auto x, auto y) -> decltype(x + y) // no const
requires(requires(){ {x + y} -> std::integral; }) // annotations after requires clause
pre { x > 0 } // proposed here
pre audit("slow for numeric code") new [&] { // (audit, new) are potential extensions
    (x > 0 && y > 0 ? as_unsigned(x) + as_unsigned(y) > as_unsigned(x) : true) &&
    (x < 0 && y < 0 ? as_unsigned(x) + as_unsigned(y) < as_unsigned(x) : true)
}
pre [&y, x, z=x] {
    // capture y by reference, x by value and copy x into z
    // check that += does the same thing as +
    (z+=y) == (x+y)
}
post [x, y] (ret) { // capture x, y by value, *explicitly*
    ret == (x + y)
}
post [&x, y] (ret) {
    // ill-formed, by-ref postconditions on value-params are meaningless for caller
    ret == (x + y)
}
{ // just to show where try-blocks go
    assert { x > 0 };
    return x += y;
}
```

4 Semantics

We specify the future in a somewhat more general manner than strictly required for the MVP, to indicate the inner workings of the future extensions.

4.1 Evaluation order

This section describes the order of evaluation *if contract checking is enabled*. If it's disabled, there is no evaluation.

4.1.1 Assertions

Assertions (any `assert`-based *correctness-specifier*) are executed as if they were immediately-invoked lambda expressions, and are therefore not a problem.

4.1.2 pre- and post-conditions

We need to make preceding **pre**-conditions protect both the *lambda-introducer* and the *correctness-specifier-body* of any subsequent *correctness-specifier*.

Therefore, **pre**-conditions are executed as is obvious: first the *correctness-specifier-introducer* (if any), and then immediately their *correctness-specifier-body*.

post-conditions are evaluated slightly differently; their *correctness-specifier-introducer* is evaluated in-sequence along with **pre**-conditions; their *bodies* are, obviously, evaluated after the function exits.

If a **pre**-condition B follows a **pre**-condition A in a function's declaration, then no part of B shall be executed before A has been proven;

If a **post**-condition P follows a **pre**-condition A in a function's declaration, then not even P 's *correctness-specifier-introducer* shall be executed before A is proven. This is to protect initialization from out-of-contract behavior.

No postcondition closure is executed before all preconditions are proven.

This means that the following execution orders are all OK:

- A, B, P
- A, A, B, B, P
- A, B, A, B, P
- A, B, P
- (prove A at compile time), B, P
- (inherit proof of A from caller precondition), B, P

4.2 post-condition reference-capture limitations in the MVP

Capturing function parameters by mutable-reference in postconditions may cause difficulties for static analysis, as some expressions containing these will require interprocedural/inter-TU analysis, which may be beyond the capabilities of a compiler. Dedicated static analysis tools should still be able to handle these, however. [P2388R2] forbids mutating function arguments.

Example (courtesy of Tomasz Kamiński):

```
int pickRandom(int beg, int end)
  post [&] (r) {
    ret >= beg &&
    ret <= end
  };
```

Given that we don't know the function body, and we could have changed **beg** and **end**, this conveys no information for static analysis (you'd have to mark **beg** and **end** **const**).

We therefore have a choice of how to start out with this proposal:

- forbid capturing parameters by mutable reference
- forbid capturing parameters by reference altogether
- do nothing and just expect degraded static analysis performance (capture-by-mutable-reference is not a problem for runtime checking)

The stated goal of feature-bijection with [P2388R2] for this paper says we should forbid reference-capture for parameters in post-conditions and only allow capture-by-value in the MVP.

4.3 Side-effect elision

This MVP presupposes that for the purposes of optimization, the compiler is allowed to either execute, or not, entire correctness specifiers, together with their closures. Subexpression elimination is only permitted under the (stricter) as-if rule.

This is because, while it should not be lippincott-discernible to the program whether a specifier was actually executed, this might only actually be true if the specifier gets to clean up after itself. In other words, the sum of the parts is assumed “pure”, the parts are not.

5 New good stuff if we pick closure-based semantics

- We get improvements in lambda-capture grammar “for free”. Once lambda-introducers get destructuring support, so do contracts, instead of inventing yet another minilanguage.
- We don’t have to re-specify anything regarding pack expansions, etc; lambda-introducers get us that, too.
- We can check time/environment-based contracts (see example below).
- Proper support for using the return-value in initializer expressions, and the ability to copy the return value so it’s not consumed.
- It’s consistent with the rest of the language, instead of inventing a yet-another minilanguage.

5.1 Stateful contracts (extension)

A yet-unserved use-case is checking whether a realtime function actually runs in the time promised; this syntax makes it easy:

```
int runs_in_under_10us()
  post [start=gettime()] { gettime() - start <= 10us };
```

Or, perhaps check we didn’t leak any memory:

```
int does_not_leak(allocator auto alloc)
  post [usage=alloc.usage(), &alloc] { usage == alloc.usage() };
```

Or, that sort actually returns a permutation:

```
void sort(auto first, auto last)
  post audit [&, input=to<vector>(first, last)]
    { is_permutation(input, {first, last}) };
```

The attribute-derived syntax does not suggest an obvious way to do this, since it doesn’t have an obvious closure.

5.2 Destructuring the return value

We need to reach for an immediately-evaluated lambda expression because we don’t have destructuring support in lambda-introducers, but that’ll change, hopefully, and when it does, we should inherit the fixes.

```
auto returns_triple()
  post (r) { [&] { auto [a, b, c] = r; return c > 0; }() }
{
  struct __private { int __a; int __b; int __c; };
  return __private{1, 2, 3};
}
```

5.3 We can have attributes appertaining to contract annotations

This one also courtesy of Andrzej Krzemiński.

```
int f(int * n)
    pre{n != nullptr}
    [[acme::audit]] pre{n >= 0};
```

A vendor of compiler extensions can always ship their own features as attributes - but this would not go quite as smoothly with almost-attributes.

6 Challenges with the attribute-derived syntax

This section explores future extensions as envisaged by [P2388R2] and previous papers.

6.1 Place for annotations like “axiom”, “new”, etc.

The best idea for where to put such markers is at the end, after a semicolon; from [P2388R2]/8:

This proposal	[P2388R2]
<pre>int f(int* p) pre {p} pre new {p > 0} ;</pre>	<pre>// after ; at end int f(int* p) [[pre: p]] // stable annotation [[pre: *p > 0; new]] // new annotation ;</pre>
<pre>int f(int* p) pre audit("allows messages") {p} pre new("2021-09-27") {p > 0};</pre>	<pre>// after : at end [[post r: r > 0: new]]</pre>
	<pre>// in braces at start [[post{new} r: r > 0]]</pre>

6.2 Referencing function arguments in postconditions

There are issues with arguments that change value during function evaluation and postconditions. They are described in [P2388R2]/6.4 and 8.1. [P2388R2] side-steps this issue by attempting to prevent referencing modified arguments, requiring that referenced arguments should be `const`-qualified (in definitions).

The ideas using the [P2388R2] syntax look like this (all from [P2388R2]/8.1):

```
// This proposal
int f(int& i, array<int, 8>& arr)
  post [i] (r) { r >= i }
  post [old_7=arr[7]] (r)
    { r >= old_7 }
```

```
// p2388r2 1)
int f(int& i, array<int, 8>& arr)
  [[post r, old_i = i: r >= old_i]]
  [[post r, old_7 = arr[7]: r >= old_7]];
```

```
// p2388r2 3)
int f(int& i, array<int, 8>& arr)
  [[post r: r >= oldof(i)]]
  [[post r: r >= oldof(arr[7])]];
```

```
// p2388r2 2)
int f(int& i, array<int, 8>& arr)
  [[post r: r >= oldof(i)]]
  [[post r: r >= oldof(arr[7])]];
```

Table 3: Another oldof example:

This proposal	P2388R2
<pre>template<class ForwardIt, class T> ForwardIt find(ForwardIt first, ForwardIt last, const T& value) post [first] (r) { distance(first, r) >= 0u } post [&last] (r) { distance(r, last) >= 0u } { for (; first != last; ++first) { if (*first == value) { return first; } } return last; }</pre>	<pre>template<class ForwardIt, class T> ForwardIt find(ForwardIt first, ForwardIt last, const T& value) [[post r: distance(oldof(first), r) >= 0u]] [[post r: distance(r, last) >= 0u]] { for (; first != last; ++first) { if (*first == value) { return first; } } return last; }</pre>

6.3 Introducing the return variable

This proposal	P2388R2
<pre>int f(int* i, array<int, 8>& arr) post [&i] (r) { r >= i };</pre>	<pre>int f(int& i, array<int, 8>& arr) [[post r: r >= i]];</pre>
	<pre>// alternative int f(int& i, array<int, 8>& arr) [[post(r): r >= 0]]</pre>

6.4 Preconditions and assertions that need copies

Table 5: [P2388R2] has no answer for preconditions that need to mutate a copy:

This proposal	[P2388R2] Does not work
<pre>int f(forward_iterator auto first, forward_iterator auto last) pre { first != last } pre [first] { std::advance(first, 1), first != last };</pre>	<pre>int f(forward_iterator auto first, forward_iterator auto last) [[pre: first != last]] // ok [[pre: std::advance(first, 1), // nope first != last]]];</pre>

6.5 Postconditions that need destructuring [when lambda-captures get it]

Table 6: Functions could conceivably have destructure-only APIs:

This proposal	[P2388R2]
<pre>auto returns_triple() post (r) { match(r) { [x, y, z] => x > y && y > z; - => false; }};</pre>	<pre>auto returns_triple() [[post r: match(r) { [x, y, z] => x > y && y > z; - => false; }]];</pre>

This syntax kind-of works, but is not proposed, and there is nowhere to specify the binding type (reference or copy?) We haven't even solved this for lambda captures, but we will, and we want to inherit the language once we do.

6.6 Multithreaded Usage

Issue courtesy of Aaron Ballman:

A potential issue with P2388R2 that is carried over into D2461R0 is with side effect operations. Given that they're unspecified, does this mean there's no safe way to write a portable contract which accesses an object shared between threads? e.g., multithreaded program where a function is passed a mutex and a pointer to a shared object; can the contract lock the mutex, access the pointee, then unlock the mutex?

With closure-based semantics, we can avoid this:

```
void frobnicate_concurrently(auto&& x)
  // closures-are-a-future-extension.disclaimer
pre [g=std::lock_guard(x)] { is_uniquely_owned(x); };
```

In this MVP, we allow the compiler to *assume there are no side-effects* to an expression for the purposes of optimisation, *but they can either all be omitted, or none may*, for a given statement, including the closure.

We therefore have a plausible RAII-based metaphor that people already understand.

6.7 Summary

- The closure-based syntax makes it obvious when values are captured, and even suggests an implementation - just put the closures on the stack before the function arguments.
- It doesn't invent another language for capturing values, which means the syntax will grow together with lambda captures.
- It makes it **obvious how to do stateful postconditions** that check before/after: the closure runs with **pre**, the body runs after return. This is far from obvious with the [P2388R2] syntax.

7 Mutation and Static Analyzers

Static analyzers should be able to handle limited mutation in order to analyze C++, and many contracts that describe function behaviour will require some mutation of a copy. Allowing copies to be made is therefore immensely useful in a contract facility.

We have assurances from at least some analyzer vendors they see no issue with allowing copies and mutation in contract annotations in the future.

8 What about abbreviated lambdas?

The post-condition syntax naturally looks like a shorthand lambda:

```
post [ closure ] ( identifier ) { conditional-expression }
```

This topic was explored in [P0573R2] by Barry Revzin, who proposed this syntax as point 2.3.

Alternative from the paper:

```
post [ closure ] ( identifier ) => conditional-expression
```

However, this doesn't work in function declarations because terminating expressions is difficult, so we like the earlier syntax better.

In this case, the default closure is [&], the parameter-type is `auto&&` and we don't need to take a stance on the `noexcept` specifier or the return type.

However, given the above, EWG should take a stance on whether that looks like a plausible set of semantics for shorthand lambdas, because we shouldn't have a yet another set of semantics for those.

9 C-compatibility

C and C++ implementations often share a set of system headers, and there will naturally be a desire to add contracts to entities in those headers.

One of the motivating reasons behind the attribute-like syntax in [P2388R2] is that a C compiler can be reasonably updated to ignore the contracts unless/until C gets Contracts as well. It's worth noting that the proposed syntax in [P2388R2] is still ill-formed for attributes, and a properly conforming C compiler that has not been updated to handle (ignore) the contracts would still issue diagnostics.

There is some debate as to whether it'd be a *good* thing if a C compiler were to still accept code that has Contracts in it when the C compiler is unaware of Contracts, and it has been noted that some implementations may simply consume all tokens in an unrecognized attribute until reaching the closing `]]`, regardless of whether the internal structure of the attribute is properly conforming.

The syntax proposed in *this* paper, however, cannot be ignored by a C compiler that is unaware of Contracts - it is unarguably ill-formed C code.

This syntax lends itself easily to conditional compilation, especially with a feature-test macro:

```
int my_func(int x)
#if __cpp_contracts /* Perhaps just __contracts to allow C to easily opt-in? */
    pre { x > 0; }
#endif /* __cpp_contracts */
{
    /* ... */
}
```

This is not a motivating difference from [P2388R2] - conditional compilation can just as easily be used to guard Contracts there; the main difference in C-compatibility between these two proposals is that [P2388R2] has a greater potential of a Contracts-unaware C compiler ignoring any contracts without a meaningful diagnostic or programmer opt-in.

10 Proposed Wording

TODO. Writing it will be an exercise, and the authors want to see if there is any enthusiasm for this at all before spending the time.

11 Acknowledgements

- *The entire WG21.* This is a huge effort, and since contracts were pulled from C++20, the group has been showing an extraordinary level of determination to get to consensus.
- *Andrzej Krzemiński*, who has been a steadfast integrator of opinion in P2338 - the MVP paper sequence. I've helped a bit, but he's been extraordinary, and also dug up more prior art and contributed examples. *I thrice presented him co-authorship, which he did thrice refuse.*
- *Tom Honermann*, who saw the interplay with function try-blocks.
- *Phil Nash*, for quite a few insightful comments, and the function-parameter syntax for return values
- *Peter Brett*, for encouraging me to drop the complex sequence of ;-separated conditions and stick to a single condition (subconditions separated by &&).
- The *BSI* for reviewing this paper early.
- *Lisa Lippincott*, for her study of stateful function contracts and all the hours she's spent explaining the point and their shape to Gašper.
- *Tomasz Kamiński*, for *also* pointing out the function parameter syntax for return values, and reminding the authors that reference-captures for non-const parameters render postconditions less useful for static analysis in the absence of the function body.
- *Ville Voutilainen*, for always connecting all of the weird bits of impact everything has on everything else.

12 References

- [N1962] L. Crawl, T. Ottosen. 2006-02-25. Proposal to add Contract Programming to C++ (revision 4).
<https://wg21.link/n1962>
- [P0573R2] Barry Revzin, Tomasz Kamiński. 2017-10-08. Abbreviated Lambdas for Fun and Profit.
<https://wg21.link/p0573r2>
- [P2388R2] Andrzej Krzemiński, Gašper Ažman. 2021-09-10. Minimum Contract Support: either Ignore or Check_and_abort.
<https://wg21.link/p2388r2>